CYPRUS INTERNATIONAL UNIVERSITY

CMPE411 Systems Programming

CHAPTER 2 - Java Threads

Assoc.Prof.Dr. Tamer TULGAR

Spring 2019-2020

Processes

- A process is a **program in execution**.

- Today, processors don't increase their speed. Instead of this, they add more cores so that the operating system can execute more than one task at a time.

- The process-based multitasking is the feature that allows you to run programs concurrently.

- Process-based multitasking allows you to execute many programs at once.

- Controlled by the Operating System’s Process Scheduler and Dispatcher.

Threads

- A multithreaded program contains two or more parts that can run concurrently
- Each part of such a program is called a thread, and each thread defines a separate path of execution.
- A thread is a single sequential flow of execution within a program.

  i.e. a thread is a subset of process.
- Thread is considered as “Light Weight Process” because it use less resource than process.
- Threads share the address space of the process but process have their own address space.
- JVM create the thread whose task is to define the main thread.
- Threads may be executed either on a multi-processor machine, or (more common) in *simulated parallel* on a single-processor machine on a time-sharing basis.

Threads and Multithreading

Advantages:

- Makes applications more responsible to input.
- Allows a server to handle multiple clients simultaneously.
- May take advantage of multiple processors.
- Using multiple threads doesn't block the user because threads are independent and you can perform multiple operations at same time.
- Threads are independent, if there occurs exception in one thread, it doesn't affect other threads.
- Threads share a common memory area.

Complications:

- Interruption of a thread may leave an object in an inconsistent state ***(safety problem)***
- A thread may block the execution of other threads **(liveness problem)**
- Complex debugging and testing processes.
- Increased difficulty level in writing a program.
- Unpredictable results.

Processes vs Threads

PROCESS	THREAD
Processes are heavyweight operations	Threads are lighter weight operations
Each process has its own memory space	Threads use the memory of the process they belong to
Inter-process communication is slow as processes have different memory addresses	Inter-thread communication can be faster than inter-process communication because threads of the same process share memory with the process they belong to
Context switching between processes is more expensive	Context switching between threads of the same process is less expensive
Processes don't share memory with other processes	Threads share memory with other threads of the same process

Multithreading

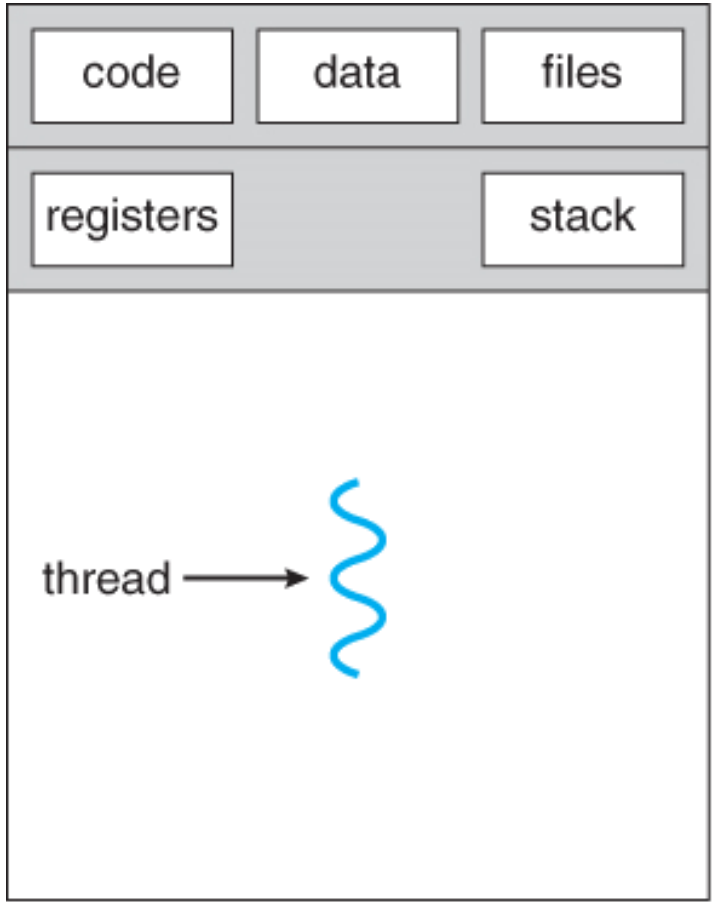
- A programming model that allows multiple threads to exists within the same context of a single process
  - Responsiveness
  - Faster execution
  - Lower resource consumption
  - Parallelization
- Java has built-in support for concurrent programming

Multithreading

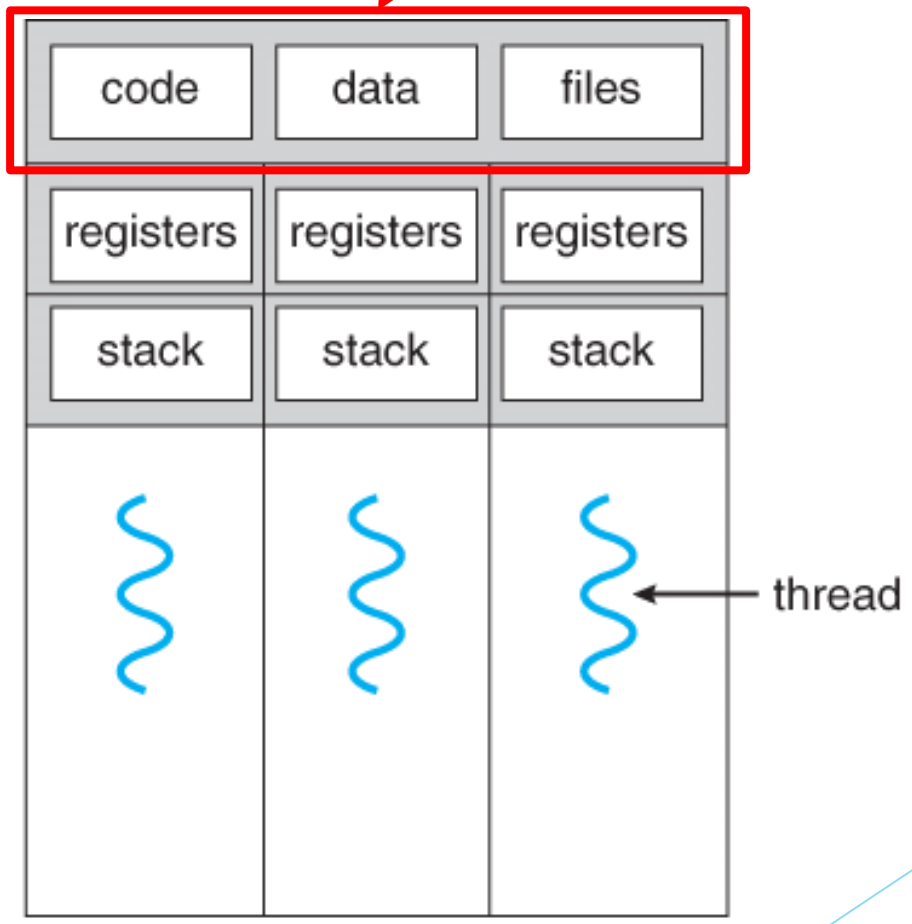
- Multithreading in java is a process of executing multiple threads simultaneously.
- Thread is basically a lightweight sub-process, a smallest unit of processing. Multiprocessing and multithreading, both are used to achieve multitasking.
- But we use multithreading than multiprocessing because threads share a common memory area.
- Threads don’t allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.
- Java Multithreading is mostly used in games, animation etc.

Multithreading

Pay attention, one common memory space is used by all of the threads of a program



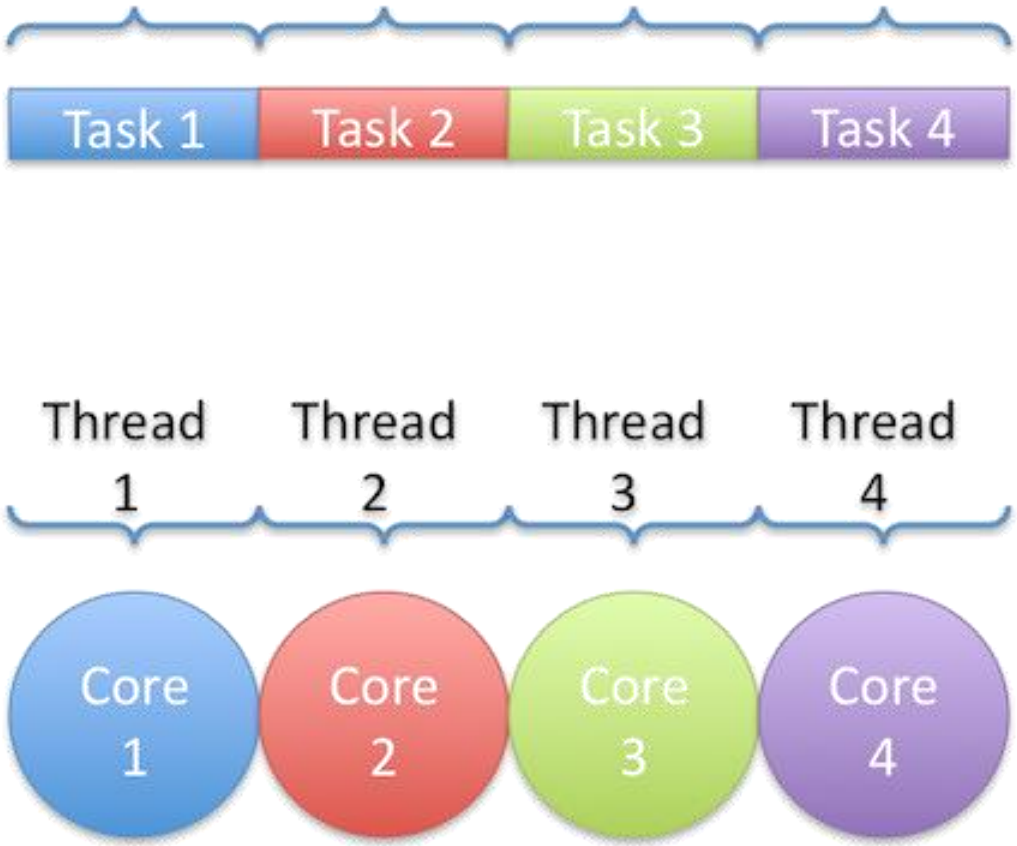
single-threaded process



multithreaded process

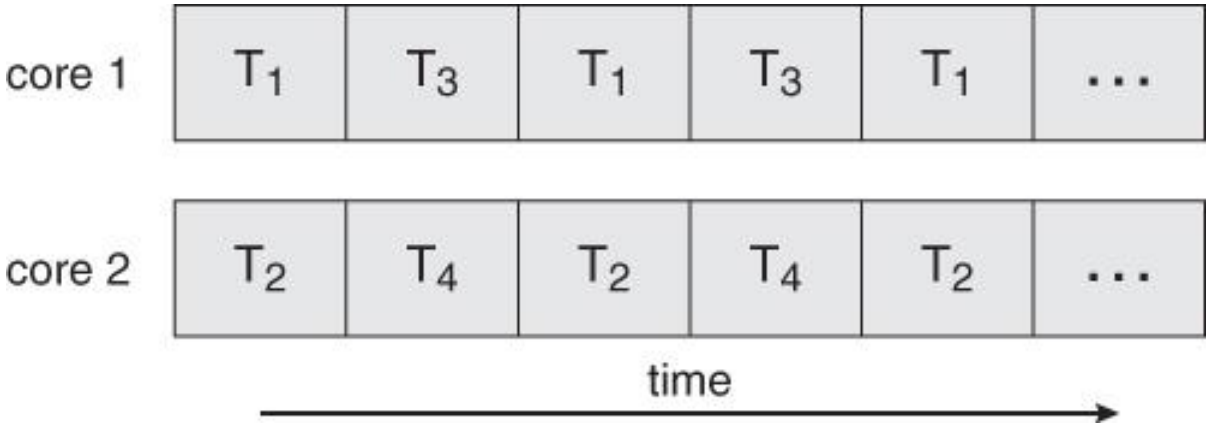
Multithreading

$$ExecutionTime \approx \sum_{i=1}^{4} Task_i$$



$$ExecutionTime \approx \underset{i}{MAX}(Task_i)$$

Multitasking



Multithreading vs Multiprocessing

Multithreading	Multiprocessing
It is a programming concept in which a program or process is divided into two or more sub programs.	It is an operating system concept in which multiple tasks are performed simultaneously.
It supports execution of multiple parts of a single program simultaneously.	It supports execution of multiple programs simultaneously.
The processor has to switch between different parts of thread or program	The processor has to switch between different programs
It is highly efficient	It is less efficient compared to multithreading
A thread is the smallest unit in multithreading	A program is smallest unit
It helps in developing efficient programs	It helps in developing efficient operating systems.

Both processes and threads provide an execution environment, but creating a new thread requires fewer resources than creating a new process.

JAVA THREADS

java.lang.Thread

java.lang.Runnable

As a JAVA programmer, you start with just one thread, called the **main thread**.

This thread has the **ability to create additional threads.**

```
class Current
{
    public static void main(String args[])
    {
        Thread t = Thread.currentThread();
        System.out.println("Current Thread: " + t);
        System.out.println("Name is: " + t.getName());
    }
}
```

```
Current Thread: Thread[main,5,main]
Name is: main
```

Creation of Threads

- Threads can be declared in one of two ways:

  1. by extending the Thread class **(NOT RECOMMENDED)**
  2. by implementing the Runnable interface **(PREFERED)**

Creation of Threads (extending the Thread class)

```
public class MyThread extends Thread
{
    public void run() {
        // the thread body!
    }

    // other methods and fields!
}
```

Creating and starting a thread:

```
(new MyThread()).start();
```

- The subclass extends *Thread* class
  - The subclass overrides the *run()* method of *Thread* class

    We overload the Thread class run() method. The run() methods defines the actual task the thread performs
- An **object instance of the subclass** can then be created
- Calling the ***start()*** method of the object instance starts the execution of the thread
  - **Java runtime starts the execution of the thread** by calling *run()* method of object instance

NEVER CALL run() method yourself. !!!!!

Call start() to start the execution of a Thread

Creation of Threads (extending the Thread class)

```
public class HelloThread extends Thread
{
    public void run()
    {
        System.out.println("Hello from a thread!");
    }

    public static void main(String args[])
    {
        Thread t = new HelloThread();
        t.start();
    }
}
```

```
(new HelloThread()).start();
```

Creation of Threads (implements Runnable)

```
public class MyRunnable implement Runnable
{
    public void run() {
        // the thread body!
    }

    // other methods and fields!
}
```

- The *Runnable* interface should be implemented by any class whose instances are intended to be executed as a thread
- The class must define run() method of no argument
  - The run() method is like main() for the new thread
- Provides the means for a class to be active while not subclassing *Thread*
  - A class that implements *Runnable* can run without subclassing *Thread* by instantiating a *Thread* instance and passing itself in as the target

Creating and starting a thread:

```
(new Thread(new MyRunnable())).start();
```

NEVER CALL run() method yourself. !!!!!

Call start() to start the execution of a Thread

Creation of Threads (implements Runnable)

```
public class HelloRunnable implements Runnable
{
    public void run()
    {
        System.out.println("Hello from a thread!");
    }

    public static void main(String args[])
    {
        Thread t = new Thread(new HelloRunnable());
        t.start();
    }
}
```

Advantage:
We can still use extends to inherit from a class !!!!

```
(new Thread(new HelloRunnable())).start();
```

Difference between the two methods

1. Inheritance Option: The limitation with "**extends Thread**" approach is that if you extend Thread, you can not extend anything else . Java does not support multiple inheritance. In reality , you do not need Thread class behavior , because in order to use a thread you need to instantiate one anyway.

2. On the other hand, **Implementing the Runnable interface** gives you the choice to extend any class you like , but still define behavior that will be run by separate thread.

3. Another difference between Thread and Runnable comes from the fact that you are extending Thread class just for run() method but you will get overhead of all other methods which come from Thread class. So, if your goal is to just write some code in run() method for parallel execution then use Runnable instead of extending Thread class.

Hence,

- When you extends Thread class, each of your thread creates unique object and associate with it.
- When you implement Runnable, it shares the same object to multiple threads.
- When you implement Runnable, you can save a space for your class to extend any other class in future or now.

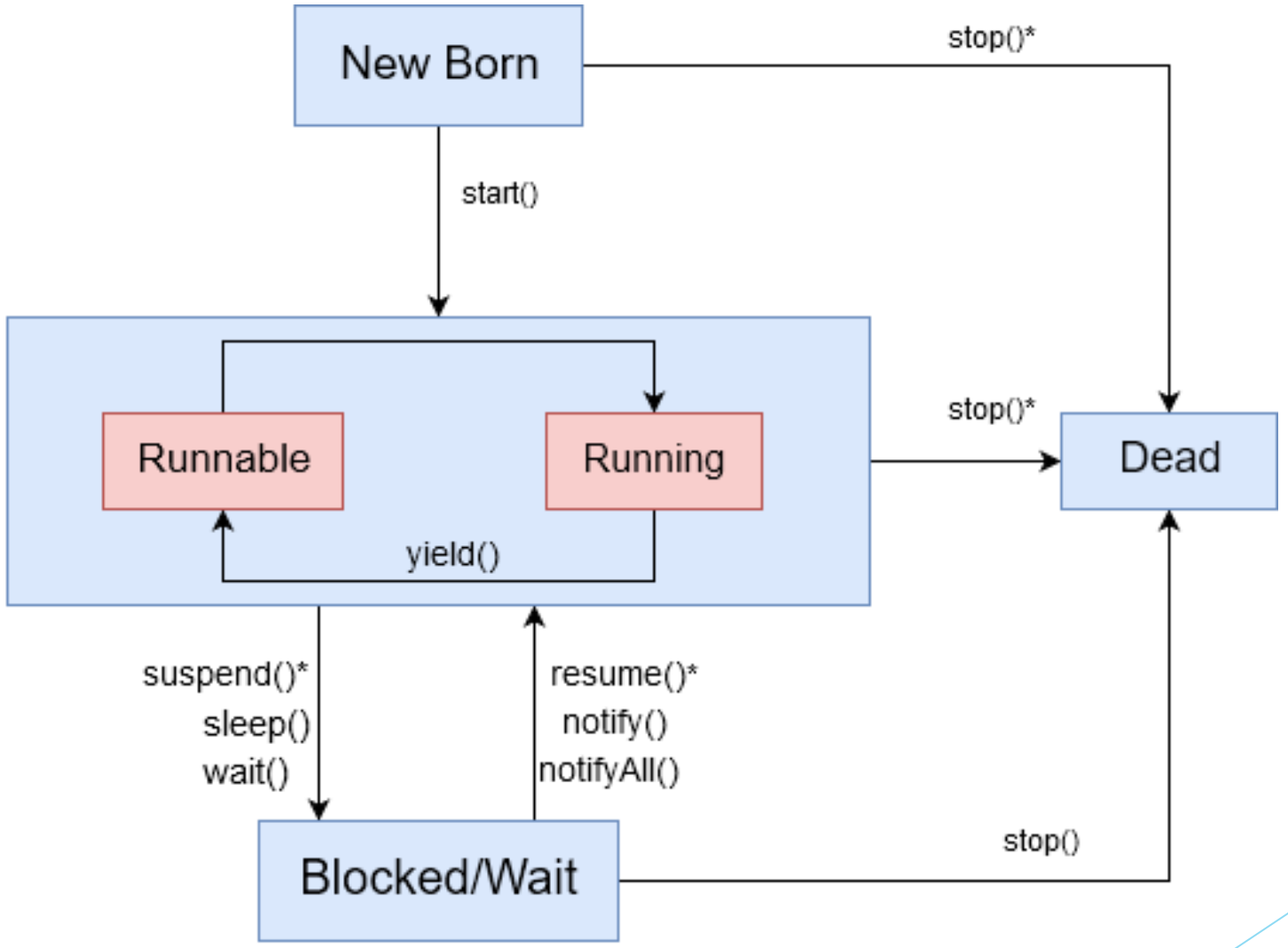
Starting Threads

Once the **start()** method is invoked.

- Resource creation, scheduling
- run method is invoked
- It may or not actually be running

SO, DO NOT CALL RUN METHOD DIRECTLY!

Life Cycle of a Thread (Thread States)



* denote deprecated Methods

Life Cycle of a Thread (Thread States)

New

The thread is in new state if you create an instance of Thread class but before the invocation of start() method.

Runnable

The thread is in runnable state after invocation of start() method, but the thread scheduler has not selected it to be the running thread.

Running

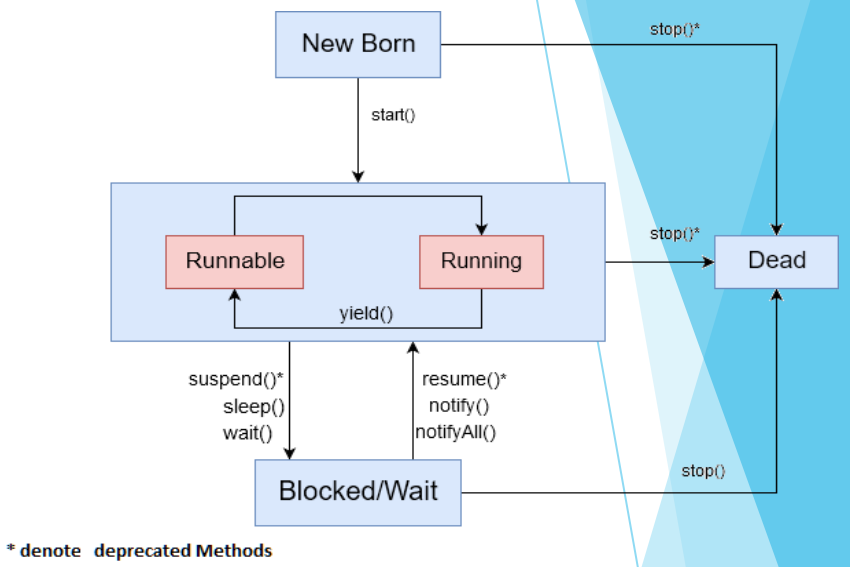
The thread is in running state if the thread scheduler has selected it.

Non-Runnable (Blocked)

This is the state when the thread is still alive, but is currently not eligible to run.

Terminated

A thread is in terminated or dead state when its run() method exits.



You can use **getState() method** to learn the current state of a thread.

What is returned will be one of the Thread.State constants.

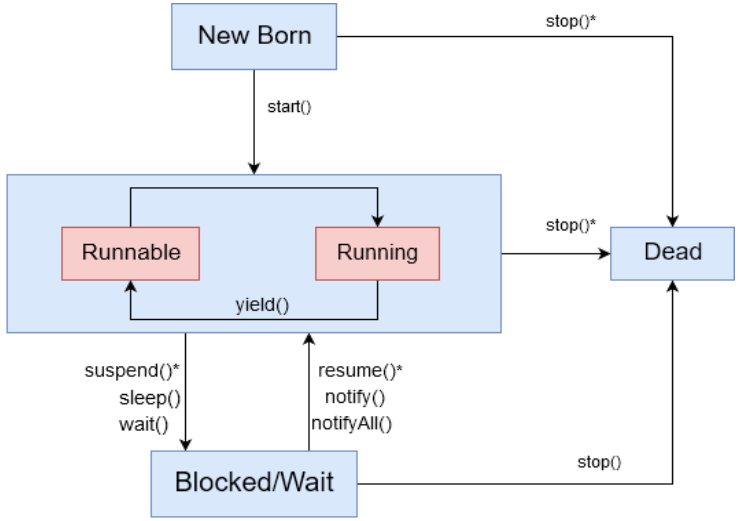
Life Cycle of a Thread (Thread States)

New threads

- Just created with the new operator. Not yet running

Runnable threads

- Once the start method is invoked.
- Resource creation, scheduling
- run method is invoked
- It may or not actually be running
- **DO NOT CALL RUN METHOD DIRECTLY!**



* denote deprecated Methods

Blocked and Waiting threads

- **Temporarily inactive**
  - A thread becomes blocked when it tries to acquire an *intrinsic* object lock
  - When a thread waits for another thread to notify of a condition, it enters the waiting state
  - The calling of a timeout parameters causes the thread to enter the timed waiting (Thread.sleep)
  - A thread waits for another thread to finish, calling the join method on it

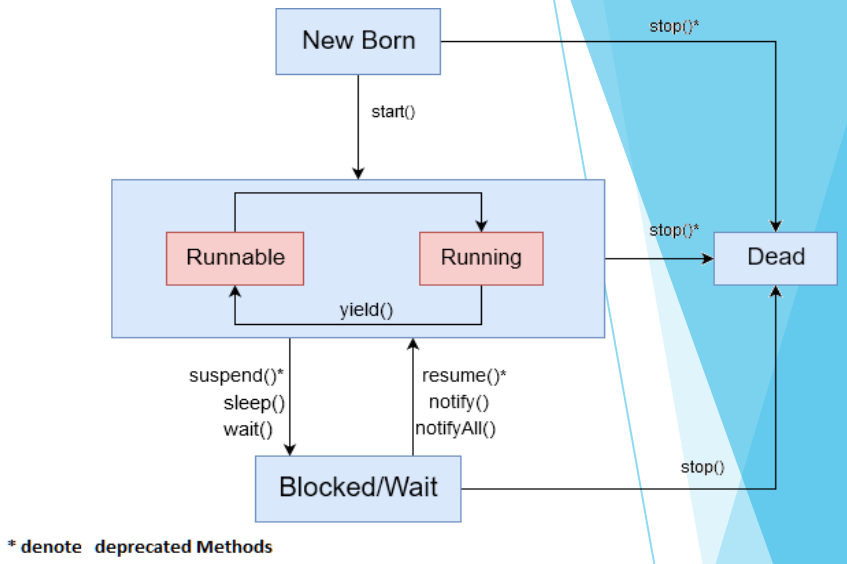
Life Cycle of a Thread (Thread States)

Terminated threads

It is not possible to reborn a thread in this state

A thread terminates when it's run method:

- Returns by executing a return statement
- After executing the last statement in method body
- If an unhandled exception occurs



ALSO,

- Threads can be **interrupted** manually (not by JVM)
- Threads can be forced to **Waiting** for another thread to finish
  - Use join() or join(long millis)
- We can implement collaborative preemption
  - Thread.yield() notifies the system that the current thread is willing to "give up the CPU" for a while.

But, we will visit these ideas in our Concurrency Chapter

Thread Priority

- Each thread is assigned a default priority of Thread.NORM_PRIORITY.
- You can reset the priority using setPriority(int priority).
- You can learn the priority using getPriority()

Some constants for priorities include:

- Thread.MIN_PRIORITY (constant of 1)
- Thread.NORM_PRIORITY (constant of 5)
- Thread.MAX_PRIORITY (constant of 10)

By default, a thread has the priority level of the thread that created it.

Don't use priorities, they are too *highly system-dependent*

Thread Interruption

yield() Method **(Sends the current thread to Runnable State for a given amount of time)**

(IN PRACTICE WE NEVER USE IT -> WE DO NOT WANT TO ENFORCE PREEMPTION)

- yield() method causes to pause current executing thread for giving the chance to remaining waiting threads of same priority.
- If there are no waiting threads or all waiting threads have low priority then the same thread will continue its execution once again.
- Thread.yield() notifies the system that the current thread is willing to "give up the CPU" for a while.
- Thread scheduler will select a different thread to run
- If no other thread is present, the statement has no effect
- Use Thread.sleep() (requires some self-computation) instead
- Use synchronization mechanisms if waiting for a process or a resource

sleep() Method **(Sends the current thread to Blocked/Waiting State for a given amount of time)**

- Interrupting a thread simply grabs its attention
  - Use Thread.sleep(long time) to suspend temporarily the thread
  - If the interrupted thread was sleeping or waiting for something, an InterruptedException is thrown
  - ...and the thread status is cleared!

Thread Interruption

join() Method **(Sends the current thread to Blocked/Waiting State until another thread terminates)**

- The Thread API contains a method for waiting for another thread to complete: the join() method.

  When you call Thread.join(), the calling thread will block until the target thread completes.

- Thread.join() is generally used by programs that use threads to partition large problems into smaller ones, giving each thread a piece of the problem

If a thread wants to wait until completing some other thread then we should go for join method.

- For example if a thread t1 wants to wait until completing t2 then “t1 has to call t2.join()”.
- If t1 executes t2.join() then immediately t1 will enter into waiting state until t2 completes.
- Once t2 completes then t1 will continue its execution.

Thread Interruption

```java
import java.util.Random;

/**
 * Creates ten threads to search for the maximum value of a
large matrix. Each thread searches one portion of the
matrix.
 */
public class TenThreads {
      final static int WIDTH = 100;
      final static int HEIGHT = 100;

      private static class WorkerThread extends Thread
      {
            int max = Integer.MIN_VALUE;
            int[] ourArray;

            public WorkerThread( int [] ourArray ) {
                  this.ourArray = ourArray;
            }

            // Find the max value in some rows of the array
            public void run() {
                  for( int i = 0; i < ourArray.length; i++ )
                        max = Math.max( max, ourArray[i] );
            }

            public int getMax() {
                  return max;
            }
      }
```

```java
      public static int[][] getBigHairyMatrix() {
            int[][] int_array = new int[WIDTH][HEIGHT];
            Random rand = new Random();

            for( int i = 0; i < WIDTH; i++ ) {
                  for( int j = 2; j < HEIGHT; j++ ) {
                        int_array[i][j] = rand.nextInt();
                  }
            }

            return int_array;
      }

      public static void main( String[] args ) {
            WorkerThread[] threads = new WorkerThread[10];
            int[][] bigMatrix = getBigHairyMatrix();
            int max = Integer.MIN_VALUE;

            // Give each thread a slice of the matrix to work with
            for( int i = 0; i < 10; i++ ) {
                  threads[i] = new WorkerThread( bigMatrix[i] );
                  threads[i].start();
            }

            // Wait for each thread to finish
            try {
                  for( int i = 0; i < 10; i++ ) {
                        threads[i].join();
                        max = Math.max( max, threads[i].getMax() );
                  }
            }
            catch( InterruptedException e ) {
                  // fall through
            }

            System.out.println( "Maxmimum value was " + max );
      }
}
```

EXERCISE : Please first implement and run this program then, rewrite it using implements Runnable

Thread Interruption

EXERCISE : Using the knowledge you got from the previous example;

Write a multithreaded JAVA application to multiply two 100 x 100 matrices.

- Like the previous example, again use 10 threads
- Assign 10 rows to each thread
- Fill your matrices with random values
- Remember you are practicing the idea that, the main thread should wait for the other 10 threads to complete, so that it can form the final result